

The background of the slide features a large, faint, light-gray circular seal of Tianjin University in the upper right corner. The seal contains the university's name in English ('TIANJIN UNIVERSITY') and Chinese ('天津大学'), along with the founding year '1895'. In the lower left corner, there is a faint, light-gray line drawing of a traditional Chinese building with a tiled roof and multiple windows.

内存管理

Memory Management



主要内容

- 虚拟存储
- 分页式内存管理
- 请求式分页
- 页表
- TLB
- 多级页表
- 倒排页表

- 内存管理方案
 - 单一连续分配
 - 分区管理
 - 页式管理
 - 段式管理
 - 段页式管理



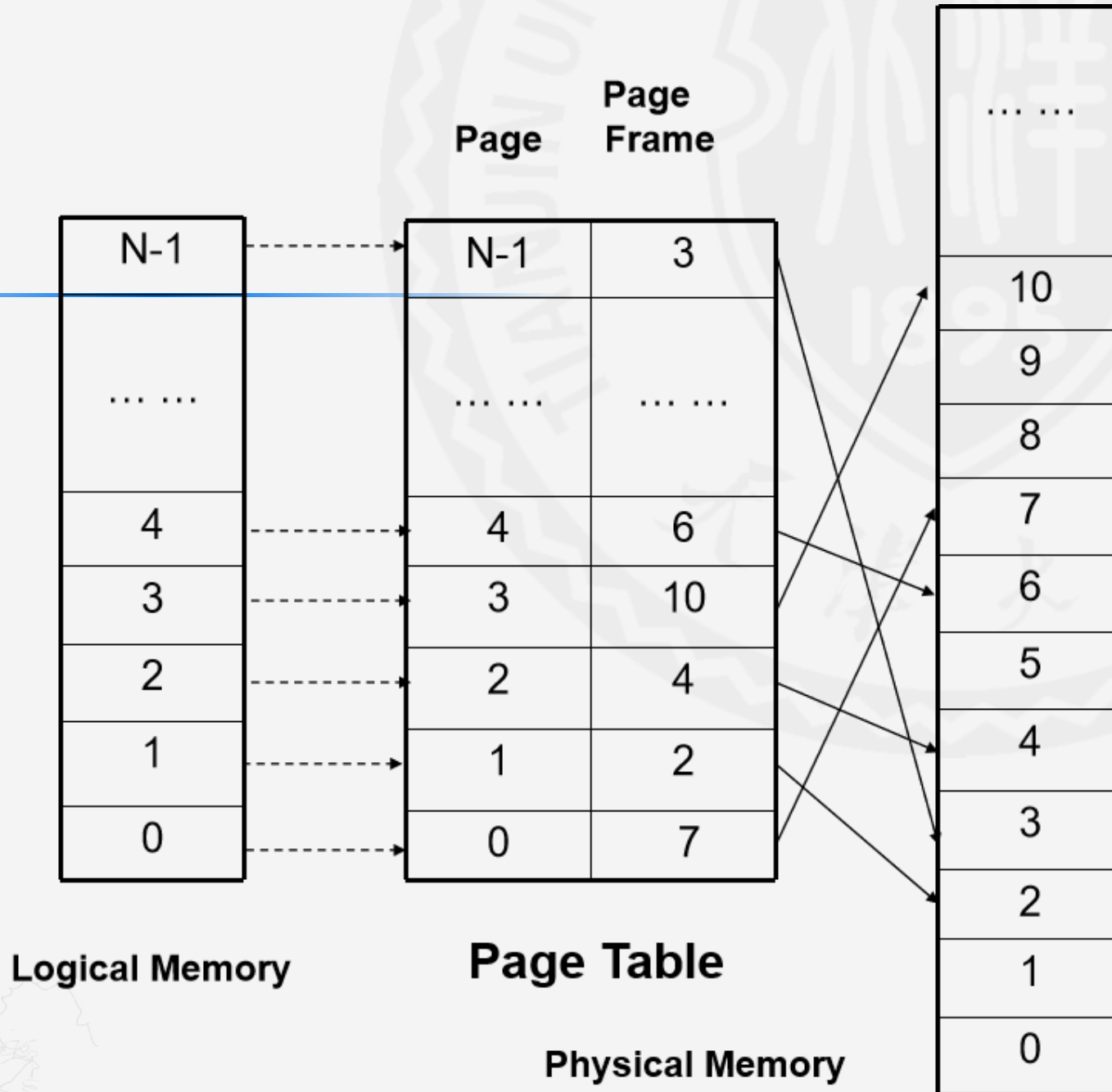
虚拟存储 (Virtual Memory)

- **虚拟存储 (Virtual Memory)** 基于 **局部性原理 (Locality Principle)**：CPU访问存储器时，无论是存取指令还是存取数据，所访问的存储单元都趋于聚集在一个较小的连续区域中。一个进程的地址空间并非一次都需要。因此，将地址空间切分成更小的部分，并且只加载需要的部分，而且，这些部分在内存中不必连续！
- 虚拟存储为进程提供比实际物理空间大得多的逻辑地址空间。分页、分段和段页式都可以实现虚拟存储，但是注意，分区式管理不可以提供虚拟存储。
- 与分区式管理中的交换（swap）的区别：交换是将处于阻塞态的进程整体换入换出，而虚拟存储是一个进程只加载需要的部分代码和数据到内存中。
- 与覆盖（overlay）的区别：虽然都是仅加载部分代码和数据到内存中，但是管理机构不同。覆盖是应用程序的开发者自行进行的操作，而虚拟存储是操作系统完成的操作，对应用程序透明。



分页式内存管理

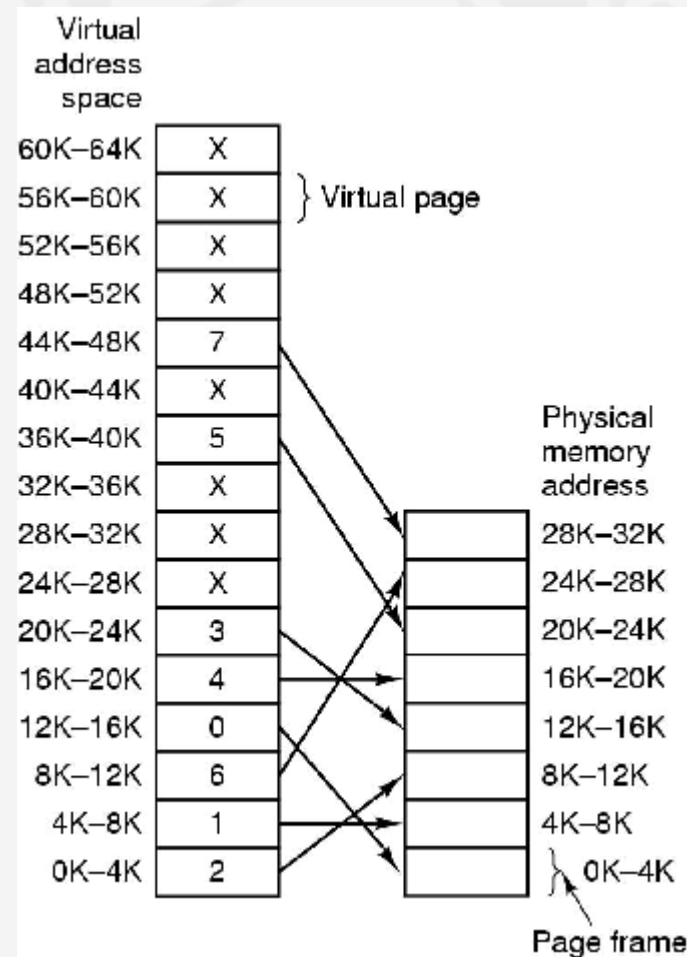
- 将程序的逻辑地址空间划分为固定大小的**页(page)**，而物理内存划分为同样大小的**页帧(page frame)**。
- 程序加载时，分配其所需的页，通过**页表 (Page Table)** 构成映射。
- 可以看出该方法的好处：没有外碎片、程序不必连续存放、便于改变程序占用空间的大小。





请求式分页

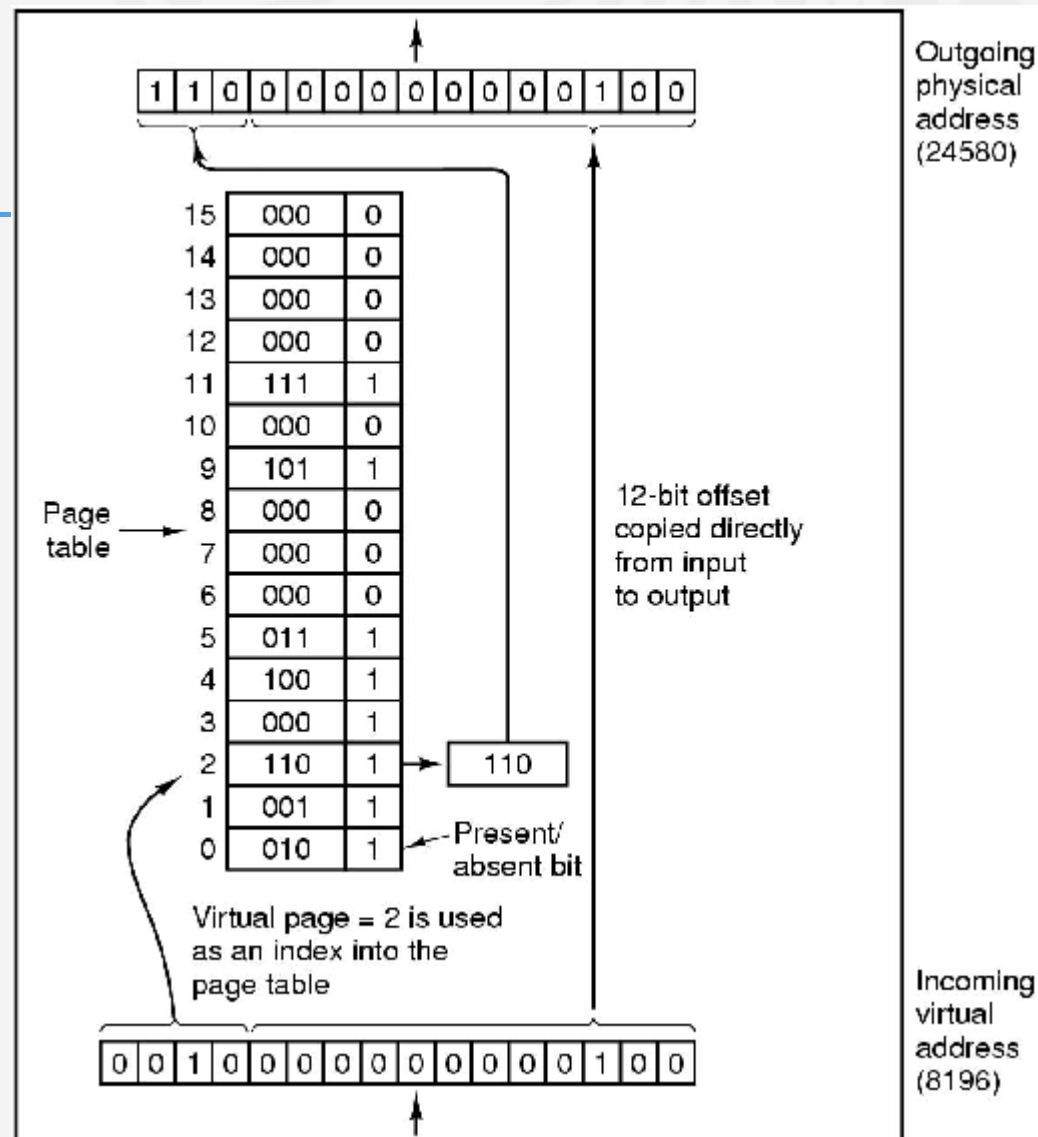
- 基于程序执行的**局部性原理 (Locality Principle)**，只需要**按需载入**需要的页到物理页帧中，暂时不需要的页就不载入。这就是请求式分页。





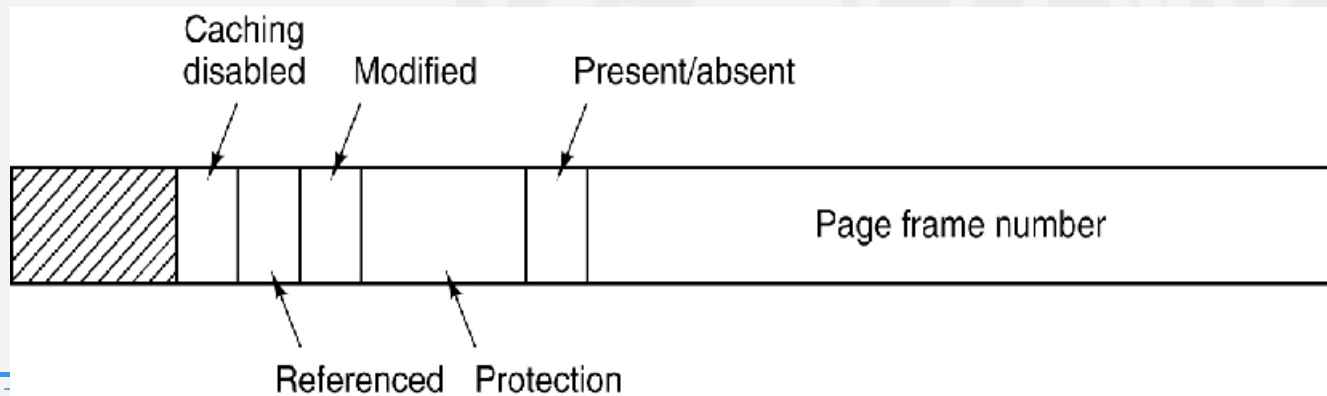
请求式分页

- 例：某系统页面大小为4KB，逻辑地址0x2004
- 页面大小4KB，说明逻辑地址中低12位为页内偏移
- 逻辑地址：0010 0000 0000 0100，查页表知
- 物理地址：0110 0000 0000 0100，即 0x6004
- 如果用十进制表示，则逻辑地址为8196
 - 页号为 $8196/4096=2$ ，页内偏移为 $8196\%4096=4$
 - 查页表，页号2对应页帧号6
 - 物理地址： $6*4096+4=24580$





页表 (Page Table)



在分页式内存管理下，每个进程应有一张**页表 (Page Table)**。页表由若干**页表项 (Page Table Entry)** 构成。页表项与页面一一对应，页表项在页表中的索引 (index) 即对应页面的页号，所以页表项中不保存页号。页表项中存放对应页面的页帧号，还有页的一些状态标志位。

- P位-Present (也被记为V位-Valid)：如果该页未装入内存，则P位为0，无页帧号。访问P为0的页面，会产生缺页中断，需要操作系统将页面调入内存后，进程方可继续运行。
- M位-Modified：如果该页从硬盘装入内存后未发生修改，则M位为0。M为0的页面说明此页面在硬盘中有备份，内存紧张时可直接将此页面从内存中移除。M位为1则需要先将页面换出到硬盘后再移除。
- R位-Referenced：当页面被访问时R位被置1。每个时钟滴答R位会被周期性清零。R位代表近期该页面被使用的情况。

15	000	0
14	000	0
13	000	0
12	000	0
11	111	1
10	000	0
9	101	1
8	000	0
7	000	0
6	000	0
5	011	1
4	100	1
3	000	1
2	110	1
1	001	1
0	010	1



页表 (Page Table)

- 在分页式内存管理下，每个进程应有一张页表。
- 页表由页表项构成，页表项的数量的计算公式：**页表项的数量 = 逻辑地址空间大小 / 页面大小**
- 页表的大小的计算公式：**页表大小 = 页表项的数量 * 一个页表项的大小**
- 页表亦存储于内存中，页表需要占页帧数量公式：**页表占页帧数量 = 页表大小 / 页面大小**
- 例：某系统页面大小4KB，逻辑地址空间4GB，页表项大小为 4B，则可推导出以下结论：
 - 页表中页表项数量为 $4\text{GB}/4\text{KB} = 1\text{M}$ 个
 - 页表大小为 $1\text{M} * 4\text{B} = 4\text{MB}$
 - 页表需要占页帧数量 $4\text{MB}/4\text{KB} = 1\text{K}$ 个



Translation Lookaside Buffers (TLB)

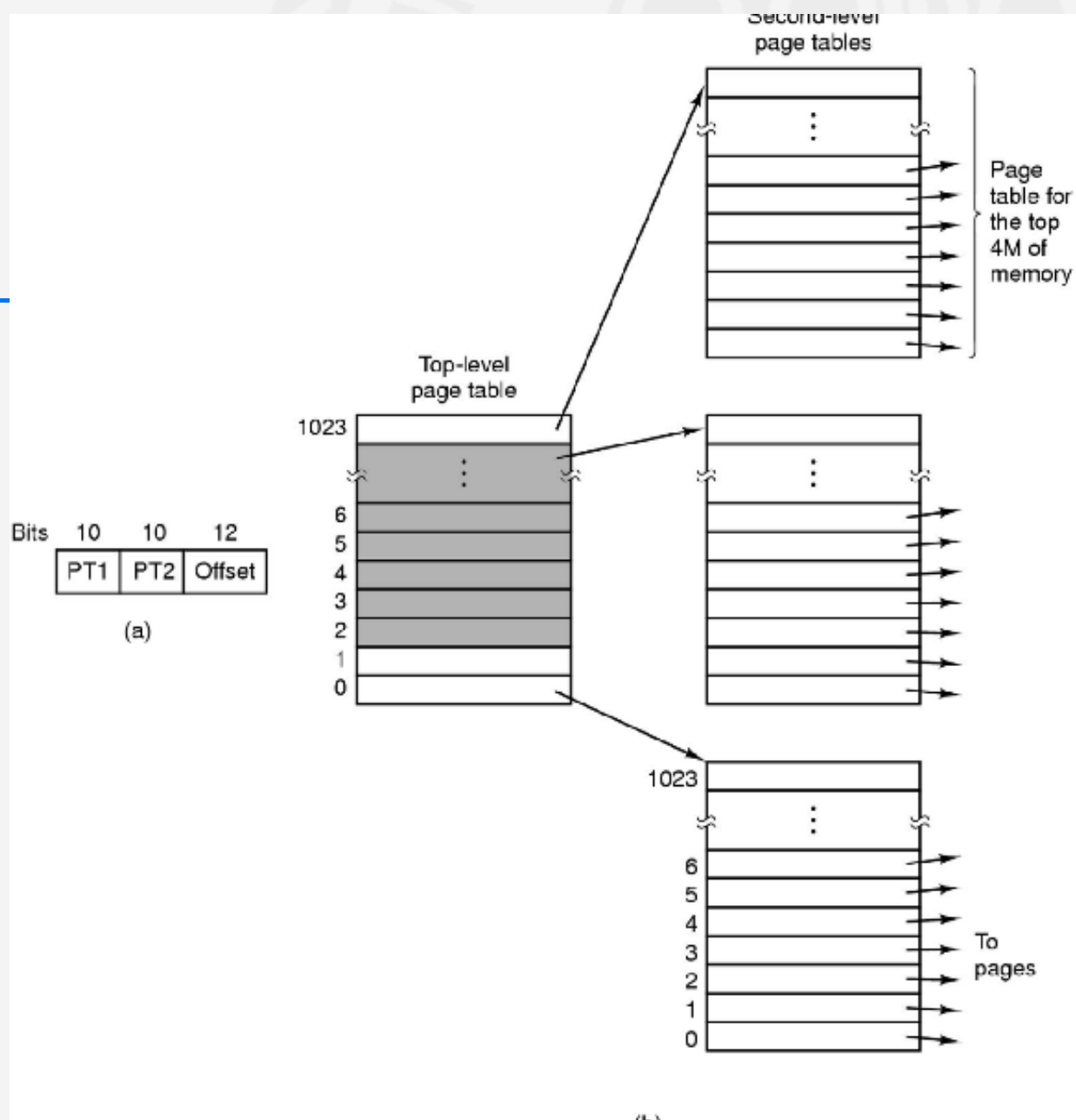
- 页表本身就在内存中，为提高地址转换速度，内存管理单元通常借助一种叫做**转译后备缓冲器（Translation Lookaside Buffer，缩写为TLB，也叫页表缓存）**的相联高速缓存（associative cache）来将虚拟页号转换为物理页号。
- TLB提速正是利用进程执行的局部性原理。TLB在缓存中，又称为“快表”，它存储常用的页表项以加速地址转换。相应的，页表被称为“慢表”。操作系统无法控制TLB。

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75



多级页表 (Multilevel Page Tables)

- **多级页表 (Multilevel Page Tables)** : 因为虚拟地址空间的大部分区域都没有使用, 因而页没有关联到页帧。为减少页表的大小, 可采用多级页表。
- 例: 4GB逻辑地址空间, 单级页表就需要4MB内存。
- 改为两级页表。将单级页表分为1024个小页表, 每个页表占4KB, 正好放入一个页帧。由于实际使用页面有限, 例如不超过3072个页, 则只需载入3个小页表即可。加上额外的一个页目录表, 只需4个页帧即可放下页表, 内存占用仅16KB。





例题

■ 某操作系统的内存管理器采用请求式分页，页面大小为1KB，逻辑地址空间为32位，物理地址空间大小为4 GB，按字节编址。页表采用多级页表，一个页表项大小为4B。TLB（快表）采用全相联映射，有4个页表项，内容如表所示。

■ (1) 该系统的页表项中，最多可以保存_____位标志位。



■ (2) 若采用多级页表，要求每级页表均可以装入一个页面内，则应该采用_____级页表较合适。



■ (3) 对逻辑地址3FFF1880H转换为物理地址的结果是_____。

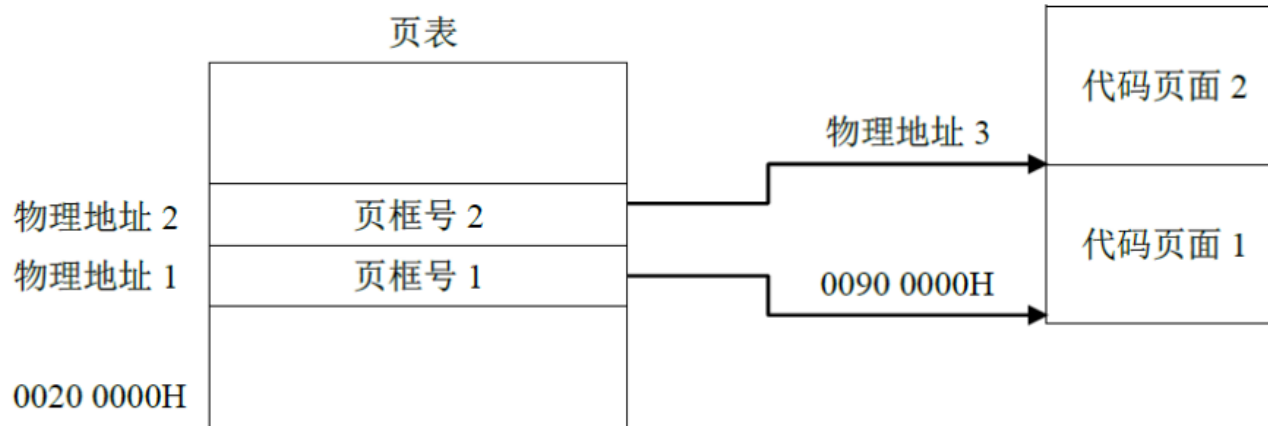
有效位	页号	页框号
0	FF180H	-----
1	3FFF1H	0F035H
1	FFFC6H	3054CH
1	03FFFH	0C153H



内存管理

Memory Management

例题



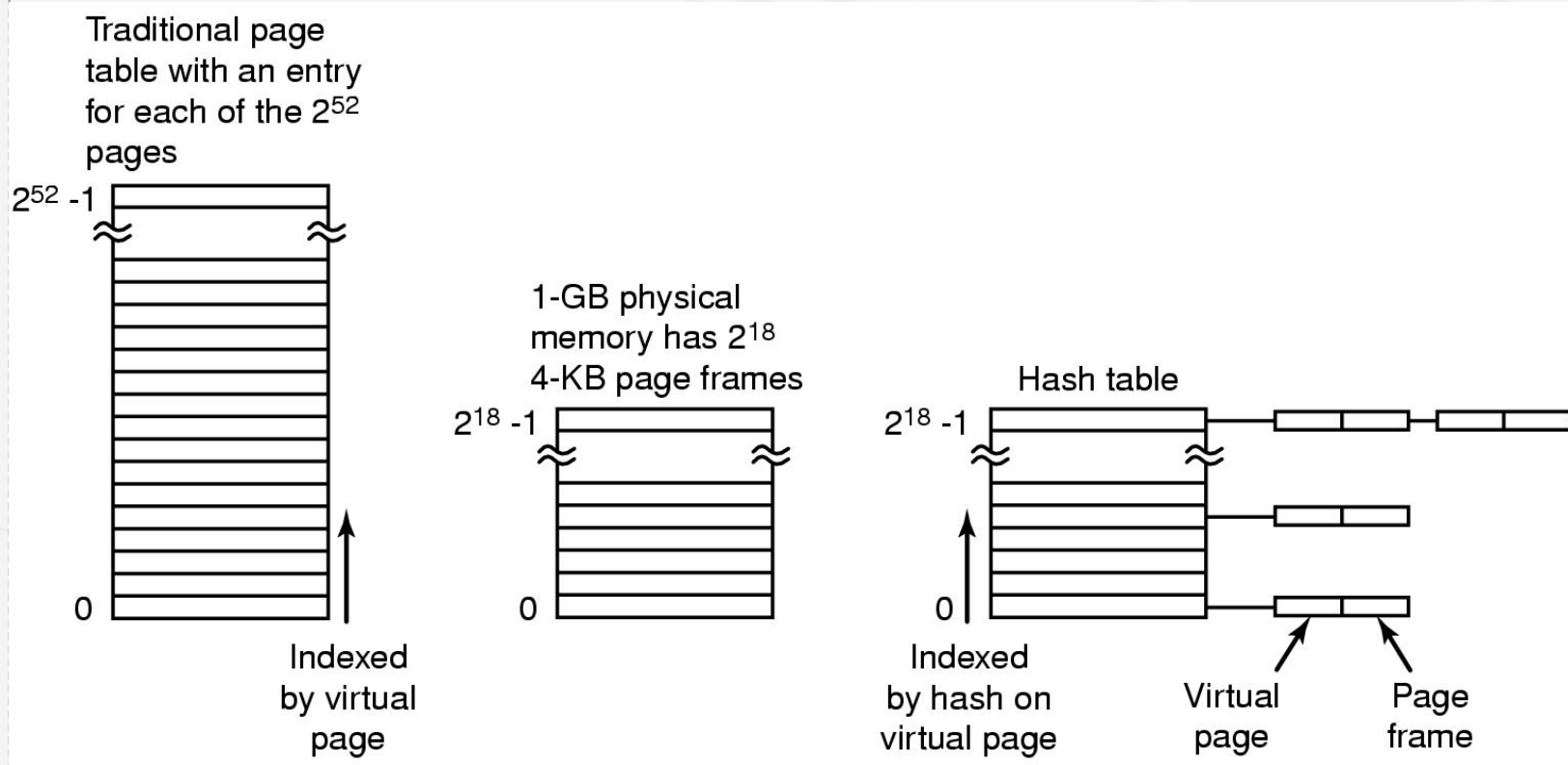
- (1) 4KB
- (2) 4MB
- (3) $LA \gg 22 \& 3FFH$
- (4) $LA \gg 12 \& 3FFH$
- (5) 16KB
- (6) 00200020H
- (7) 00200024H
- (8) 00900H
- (9) 00901H
- (10) 00901000H

- 某计算机主存按字节编址，逻辑地址和物理地址都是32位，页表项大小为4字节。请回答下列问题。
- (1) 若使用一级页表的分页存储管理方式，逻辑地址结构为：页号（20位）+ 页内偏移量（12位）。则页的大小是__ (1) __。页表最大占用空间为__ (2) __。
- (2) 若使用二级页表的分页存储管理方式，逻辑地址结构为：页目录号（10位） + 页表索引（10位）+ 页内偏移量（12位）。设逻辑地址为LA，则其对应的页目录号的表达式__ (3) __和页表索引的表达式__ (4) __。若该进程共用到了3072个页，则此时此二级页表占用的总空间最小为__ (5) __。
- (3) 采用（1）中的分页存储管理方式，一个代码段起始逻辑地址为0000 8000H，其长度为8 KB，被装载到从物理地址0090 0000H开始的连续主存空间中。页表从主存0020 0000H开始的物理地址处连续存放，如下图所示（地址大小自下向上递增）。则该代码段对应的两个页表项，物理地址1是__ (6) __，物理地址2是__ (7) __；这两个页表项中的页框号1是__ (8) __，页框号2是__ (9) __；以及代码页面2的起始物理地址3是__ (10) __。



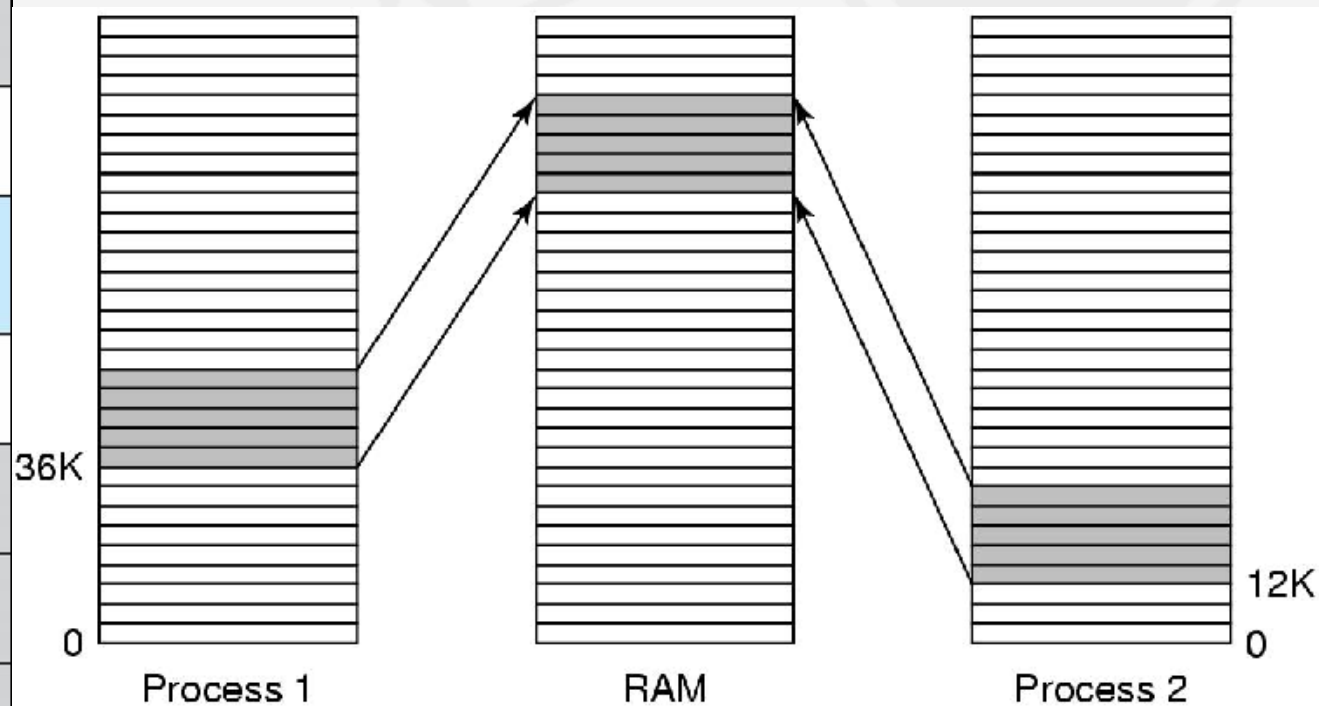
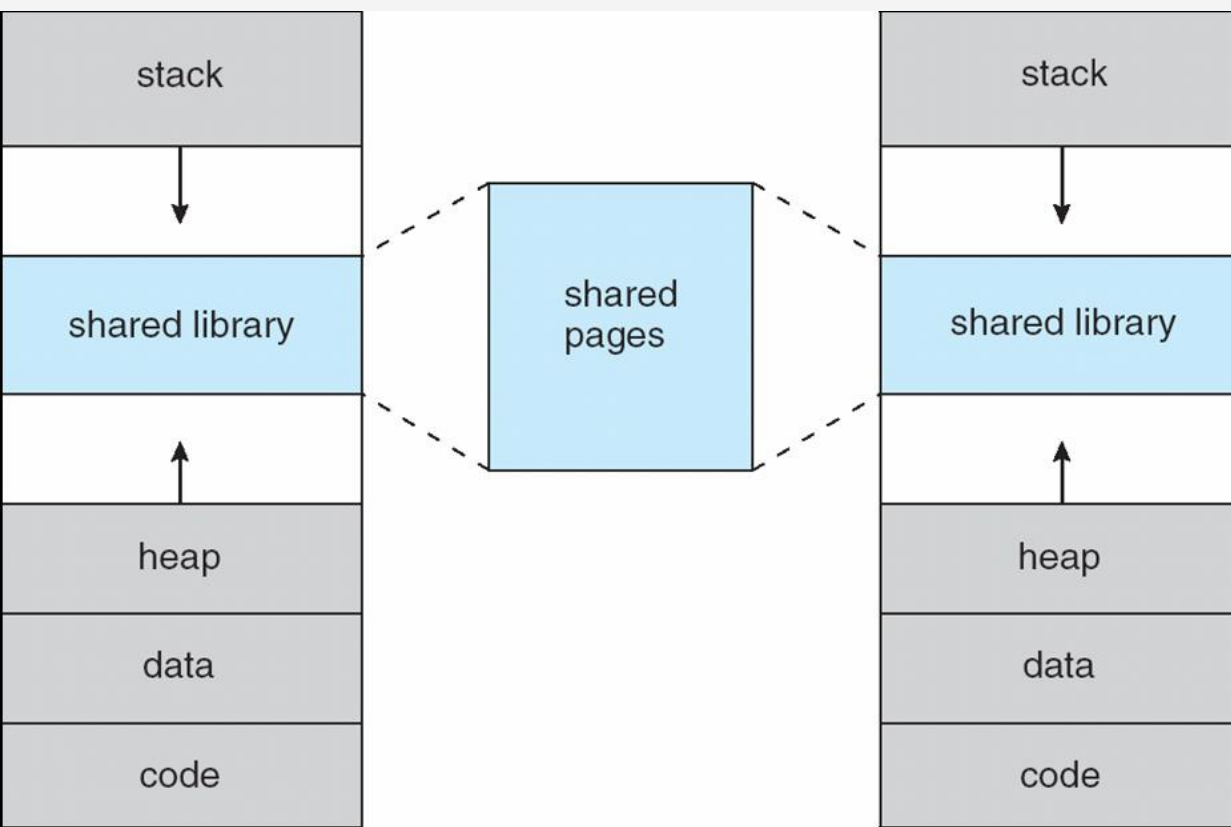
倒排页表 (Inverted Page Tables)

- 在64位操作系统中，因为有64位地址线，所以页表的大小可能非常非常大，虽然分级页表可以不必加载全部页表，IA-32、IA64系统一般使用四级页表来处理，而在PowerPC等体系中则使用倒排页表来解决这个问题。
- 与传统页表的区别：使用页框号而不是虚拟页号来索引页表项。
- 需要使用hash表来索引。





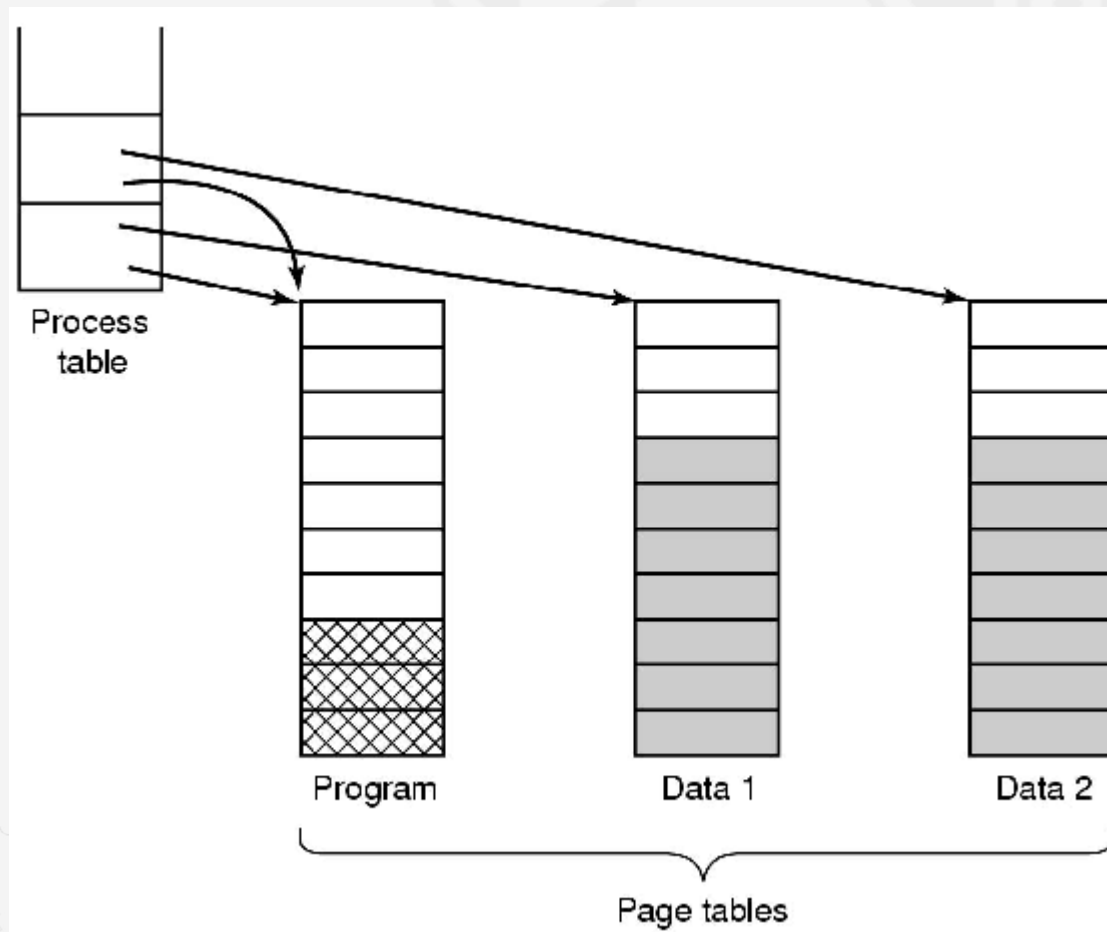
虚拟内存中使用共享库函数





虚拟内存中共享代码

- 几个不同的用户同时运行同一个程序很常见，由于避免了在内存中有一个页面的两份副本，共享页面效率更高。只读的页面（诸如程序文本）可以共享，但是数据页面则不能共享。



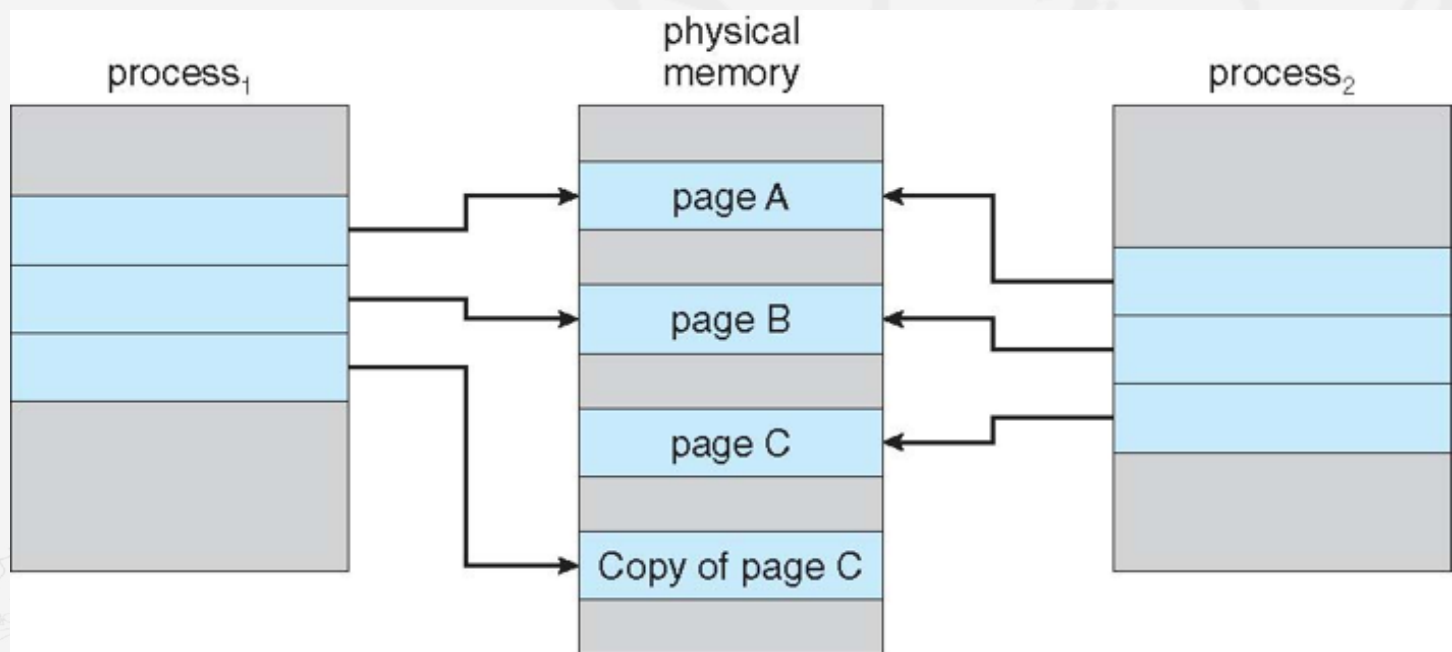
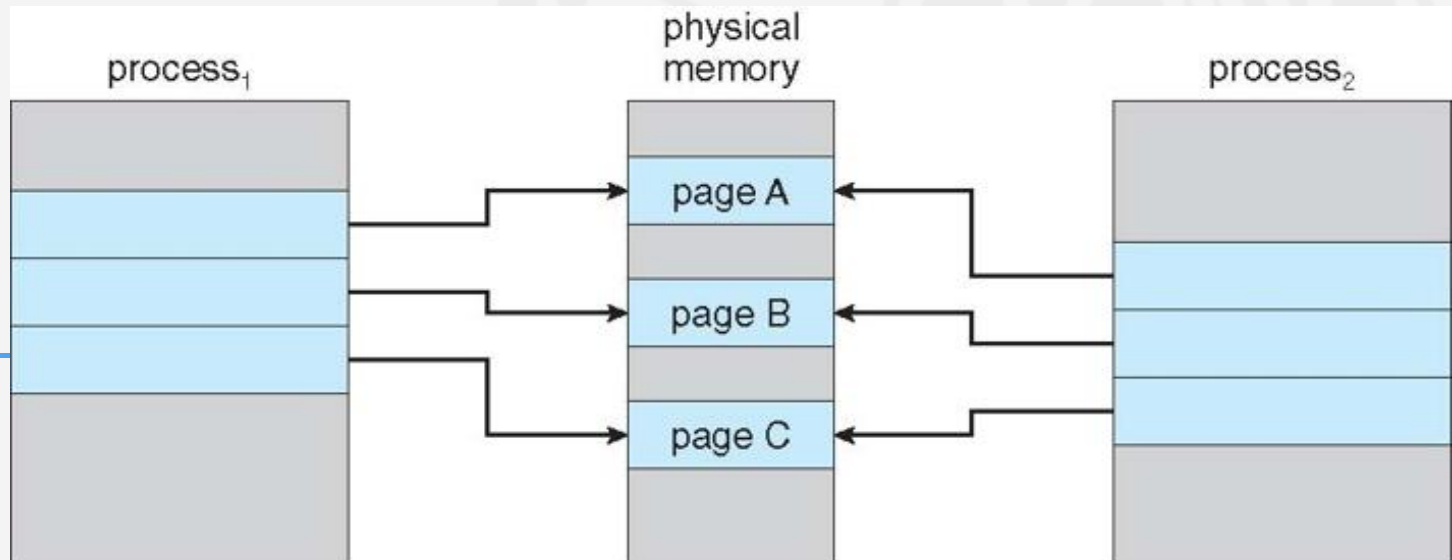


内存管理

Memory Management

写时复制

- **写时复制 (Copy-on-Write, COW)** 允许父进程和子进程最初共享内存中的相同页面（上图，进程1和进程2为父子进程关系）。
- 如果任一进程修改了共享页，则才进行复制（如图，进程1修改了页面C）。
- COW允许更高效地创建进程，因为只有被修改的页才会被复制。

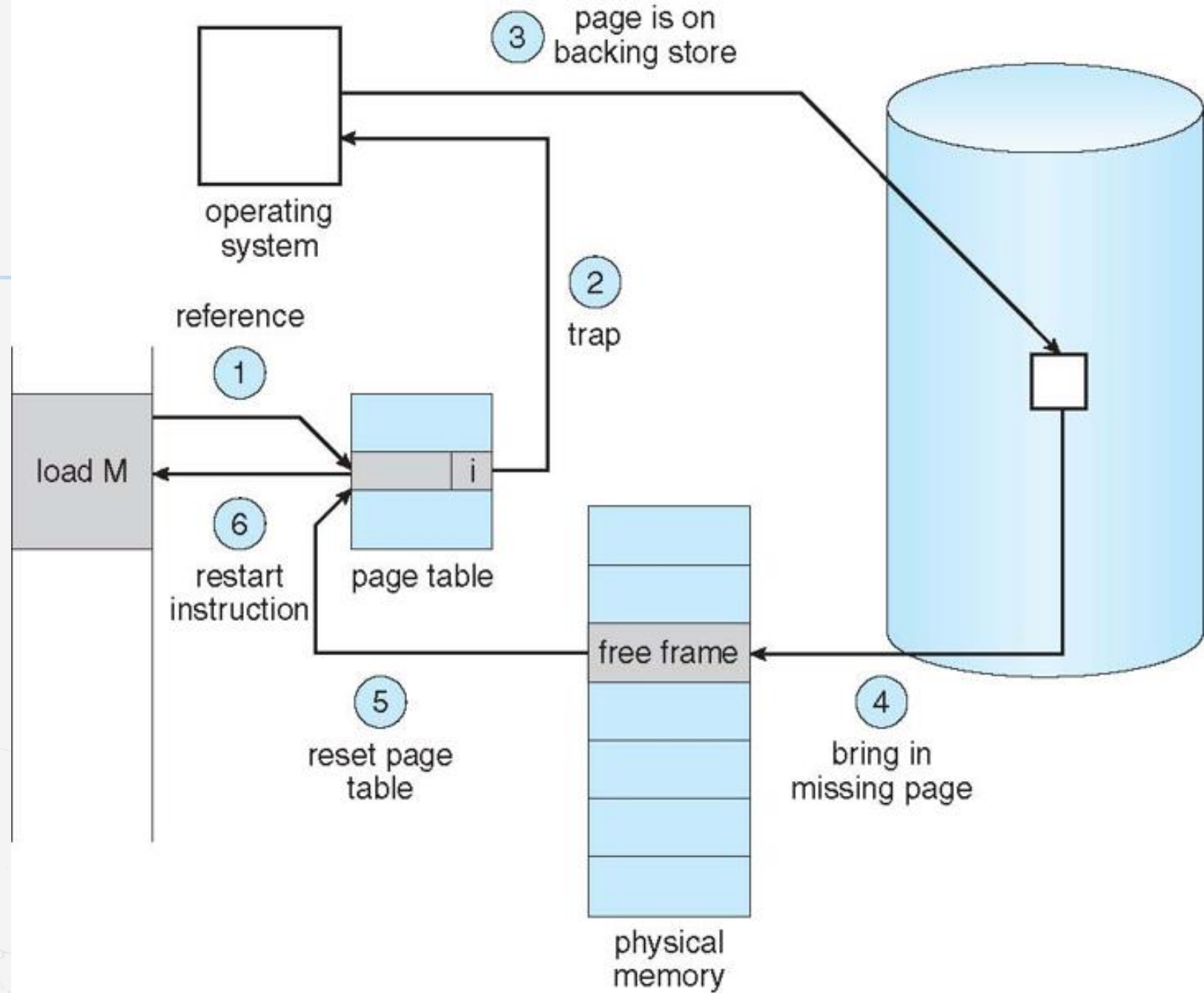


内存管理

Memory Management

处理缺页中断的过程

- 注意：步骤(4)：如无空闲页帧，则需启动页面置换。
- 步骤(5)：同时更新了TLB快表和页表。
- 步骤(6)：缺页中断处理完后，是重新执行被中断的指令。





课后阅读

- 《操作系统概念 OSC（第9版）》第8章、第9章。
- 《现代操作系统 MOS（第4版）》第3章。
- 《深入理解计算机系统 CSAPP（第3版）》第6章、第9章。